

Introduction to System Programming Final Exam Review

Outline

This outline is structured to cover the key concepts from the textbook (Chapters 1-13), the instructor's emphasis on programming concepts and practical applications (e.g., collections, streams, tests), and your unfamiliar topics. It prioritizes understanding over exact syntax—use pseudo-code where helpful. Focus on why concepts matter, how they operate, and examples. Review preconditions as an overarching theme for method reliability.

1. Core Programming Concepts (From Book Chapters 1-13)

- **Class vs. Object:** A class is a blueprint defining structure (fields, methods) and behavior; an object is an instance of a class with actual data. Example: Class `Car` defines attributes like `color`; object `myCar = new Car()` has `color = "red"`.
- **Visibility (Access Modifiers):** Private hides internal details for security/encapsulation; public exposes for use. Importance: Prevents unintended changes, promotes modular design. Protected for inheritance; default (package-private) for same-package access.
- **Preconditions:** Conditions that must be true before a method runs (e.g., input not null). Importance: Ensures reliability; violations lead to errors. Example: Method `divide(int a, int b)` precondition: `b != 0`.
- **Null vs. Empty** (Unfamiliar #1): Null means no reference (uninitialized or absent value); empty means initialized but with no content (e.g., empty string `""` or empty list). Importance: Null causes `NullPointerException`; check with `== null`. Empty: Use methods like `isEmpty()` for collections/strings.
- **Abstraction and Modularization** (Unfamiliar #2): Abstraction hides complexity, showing only essentials (e.g., abstract classes/interfaces). Modularization breaks code into reusable modules (e.g., separate classes for concerns). Importance: Improves readability, reuse; reduces bugs.
- **Accessor and Mutator** (Unfamiliar #3): Accessors (getters) retrieve field values (e.g., `getName()`); mutators (setters) modify them (e.g., `setName(String)`). Importance: Control access, enforce invariants (e.g., validate input in setter).
- **Encapsulation** (Unfamiliar #6): Bundling data and methods, hiding internals via private fields and public accessors/mutators. Importance: Protects data integrity, allows internal changes without affecting users.
- **Cohesion** (Unfamiliar #5): How focused a class/module is on a single responsibility. High cohesion: One class does one thing well (e.g., `Calculator` only computes). Low: Scattered duties. Importance: Easier maintenance.
- **Coupling** (Unfamiliar #4, part of): Degree of interdependence between classes. Low coupling: Classes interact minimally (e.g., via interfaces). High: Tight dependencies make changes hard. Importance: Promotes flexibility; aim for low coupling/high cohesion.
- **Static** (Unfamiliar #4, part of): Belongs to class, not instances (e.g., static methods/variables). Importance: Shared across objects; no `this` reference. Example: `Math.PI` is static.
- **Refactoring** (Unfamiliar #7): Improving code structure without changing behavior (e.g., extract method, rename variables). Importance: Enhances readability, reduces duplication; tools like IDEs help.
- **Casting, is-a Relation, instanceof** (Unfamiliar #8): Casting converts types (e.g., `(SubClass) superObj`); is-a: Inheritance relation (`SubClass` is-a `SuperClass`). `InstanceOf` checks type (e.g., `obj instanceof Class`). Importance: Polymorphism; avoid `ClassCastException` by checking `instanceof` first.

2. Collections and Streams (Practical Programming Focus)

- **Collections Basics:** Interfaces like List (ordered, duplicates ok, e.g., ArrayList), Set (unique, e.g., HashSet), Map (key-value, e.g., HashMap). Operations: add, remove, iterate. Not inherently streams—convert via `.stream()`.
- **Using/Combining Collections:** Solve problems like storing unique items (Set), key-based lookup (Map), or nested (e.g., `Map<List<String>>`). Example: Track student grades—`Map<String, List<Integer>>`.
- **Streams:** For functional processing. Key: Streams are lazy, one-time use; operations chain (intermediate return streams, terminal produce results).
 - Obtain: `collection.stream()`.
 - Operations: Filter (takes predicate, returns stream: `stream.filter(x -> x > 0)`); map (transform: `map(x -> x*2)`); reduce/collect (terminal: `collect(Collectors.toList())`).
 - Pseudo-code example: Translate loop to stream—From `for (int i : list) if (i even) sum += i;` to `list.stream().filter(i -> i % 2 == 0).reduce(0, (a,b) -> a+b);`.
- **Importance:** Streams promote concise, parallelizable code; know chaining and lambda parameters (e.g., filter's boolean function).

3. Testing and Unit Tests (Practical Programming Focus)

- **Unit Test Components** (Unfamiliar #4, part of): Setup (initialize objects), Act (call method), Assert (verify output, e.g., `assertEquals`). Frameworks like JUnit: `@Test` annotations.
- **Writing Tests:** For functionality—e.g., test edge cases (null, empty), preconditions. Example: For `add(int a, int b)`, test `assertEquals(5, add(2,3))`; and exception for invalid input.
- **Importance:** Catches bugs early; low coupling aids isolated testing.

4. Inheritance, Polymorphism, and Related Concepts

- **From Book:** Overriding, super, abstract classes, interfaces. Relate to is-a (inheritance) vs. has-a (composition).
- **Examples:** Animal class with Dog subclass—is-a relation allows polymorphism (Animal dog = new Dog(); dog.makeSound() calls Dog's override).

5. Problem-Solving Exercises (Instructor's Examples)

- **Collections Problems:** E.g., Remove duplicates: Use Set. Combine: List of Maps for grouped data.
- **Stream Translations:** Convert imperative (loops) to declarative (streams). Practice: Filter even numbers, sort, collect to set.
- **Tests for Programs:** Given code, write tests covering normal/precondition violation cases.
- **General Tips:** Focus on "how things operate"—e.g., why use stream over loop (readability), why encapsulation (data hiding).

Review Strategy

- Read Chapters 1-13: Skim familiar, deep-dive unfamiliar (e.g., OOP in Ch. 5-8, collections in Ch. 10-12).
- Practice: Write pseudo-code for streams/tests; explain concepts verbally (e.g., why low coupling?).
- Mock Exam: Mix concept questions (define + example) with coding (e.g., "Write a stream to filter positives from a list").